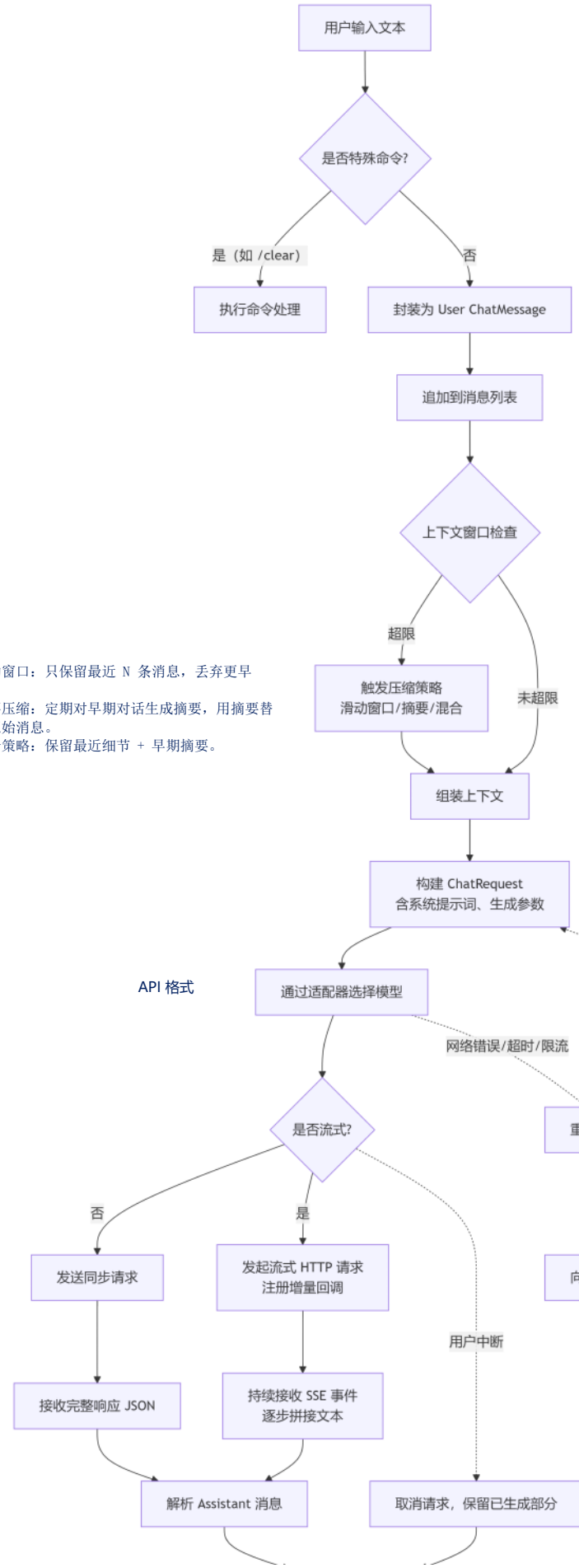
002 会话流程与缓存

2026年9月2日 21:18

普通会话：



无状态模型与消息列表
大模型 API 本身不保存任何对话状态，每次请求都是独立的。因此，为了实现多轮对话，应用必须维护一个消息列表，并在每次请求时将完整历史发送给模型。模型通过阅读历史消息来“记住”之前的对话内容。

消息列表中的角色（System，User，Assistant）帮助模型区分信息来源，从而正确生成回复。系统提示词通常固定不变，用于设定模型的行为

角色为 User，内容为文本

追加到当前会话的消息列表（messages）末尾。
消息列表可能已经包含之前的对话记录（由 Assistant 和 User 消息交替组成），以及可能存在的 System 提示词

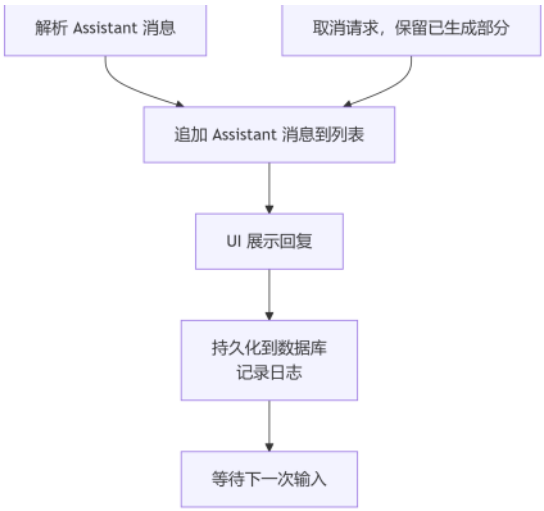
- 滑动窗口：只保留最近 N 条消息，丢弃更早的。
- 摘要压缩：定期对早期对话生成摘要，用摘要替换原始消息。
- 混合策略：保留最近细节 + 早期摘要。

包括历史消息和当前输入

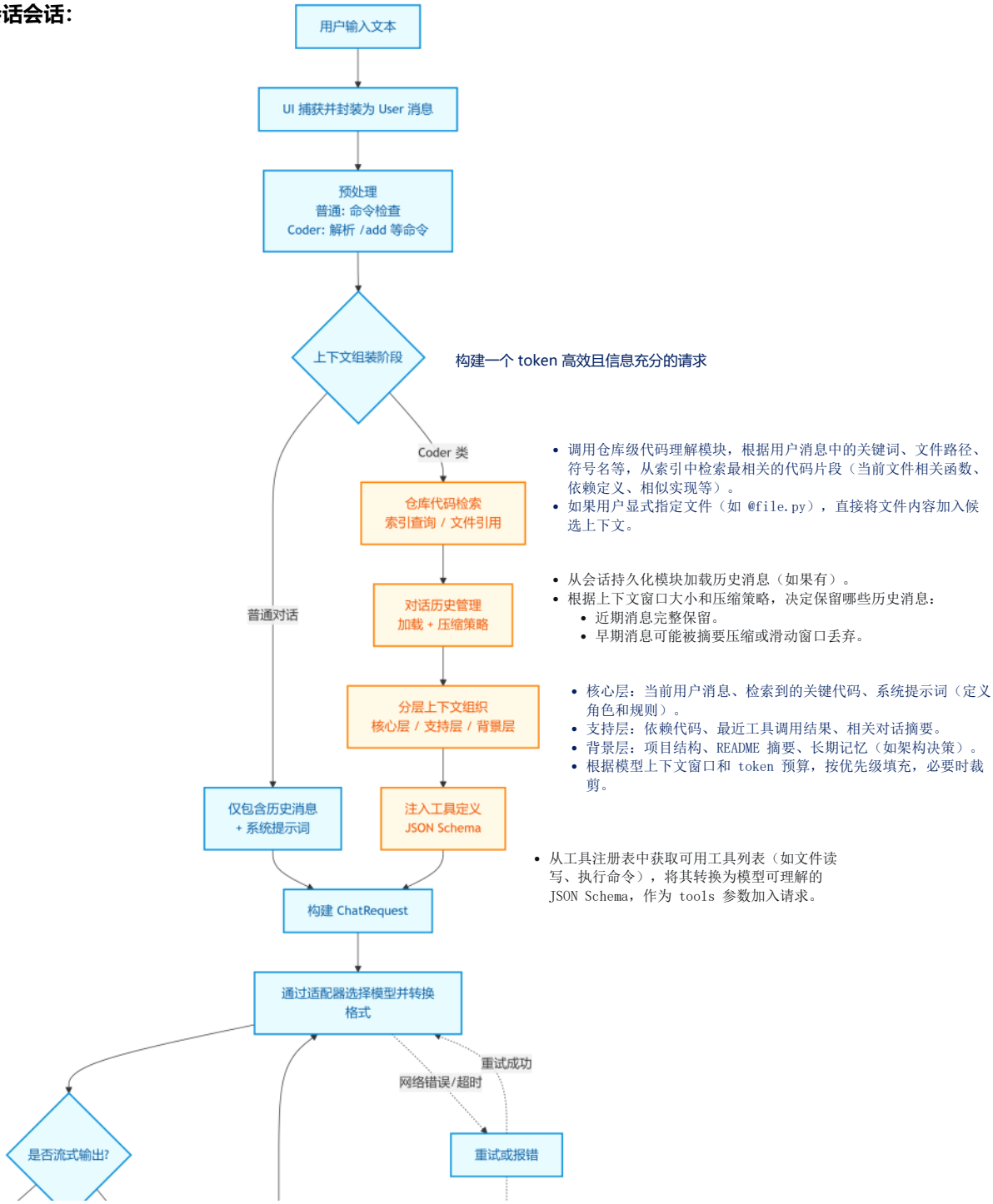
设定模型角色、行为规范

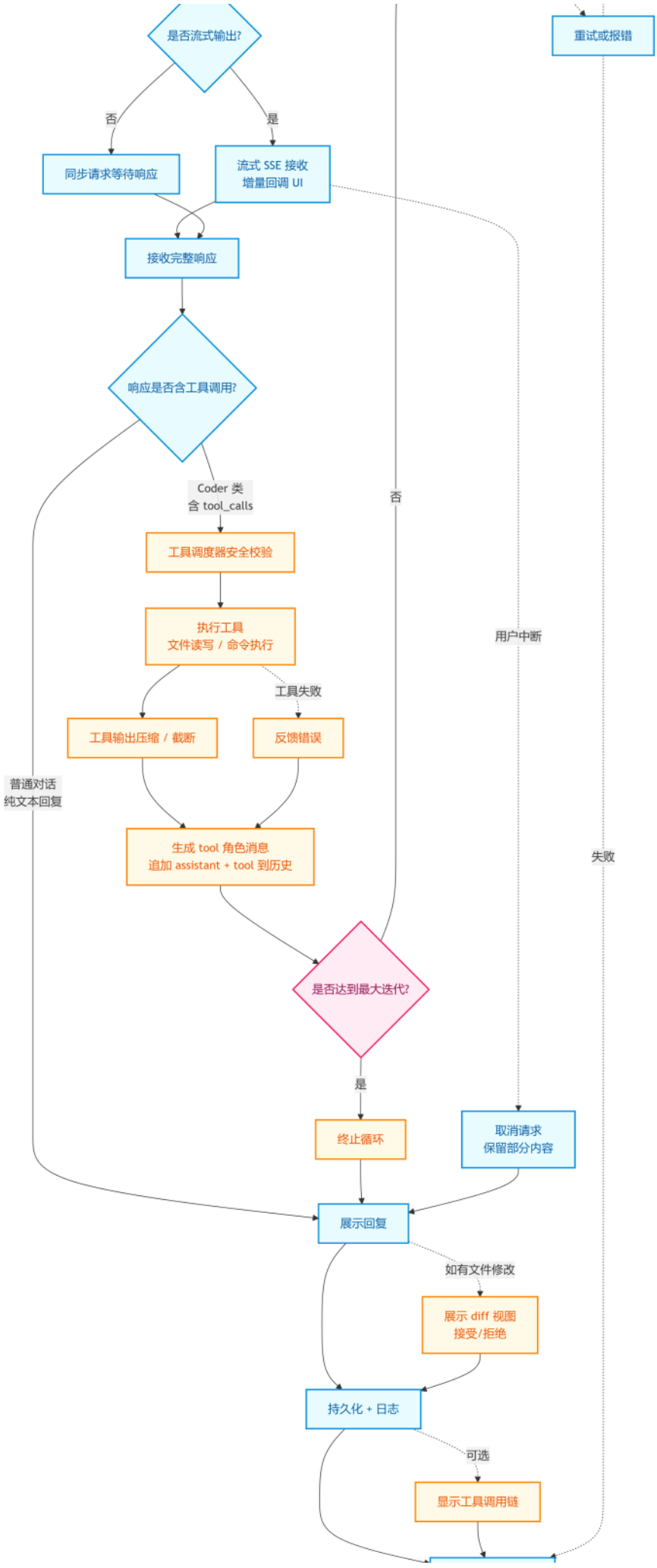
max_tokens、stream

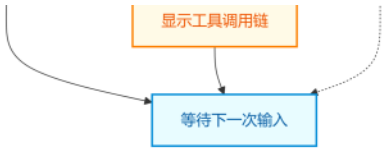
API 格式



Coder 类 AI Harness 会话会话：




是否流式输出?
否
是
同步请求等待响应
流式 SSE 接收
增量回调 UI
接收完整响应
响应是否含工具调用?
Coder 类
含 tool_calls
普通对话
纯文本回复
工具调度器安全校验
执行工具
文件读写 / 命令执行
工具失败
工具输出压缩 / 截断
反馈错误
生成 tool 角色消息
追加 assistant + tool 到历史
是否达到最大迭代?
否
是
终止循环
展示回复
如有文件修改
展示 diff 视图
接受/拒绝
持久化 + 日志
可选
显示工具调用链
重试或报错
失败
用户中断
取消请求
保留部分内容



大模型会话缓存：

1. 大模型推理基础：自回归生成与 KV Cache

1.1 自回归生成

大语言模型（Transformer Decoder）按顺序生成 token：

- 输入一个 token 序列（提示词），模型输出下一个 token 的概率分布。
- 采样得到一个 token，将其附加到序列末尾。
- 将新的序列作为输入，继续生成下一个 token。
- 重复直到生成结束符或达到最大长度。

1.2 KV Cache

Transformer 的每一层都有自注意力机制，需要计算 Query (Q)、Key (K)、Value (V)。在生成第 t 个 token 时，模型需要计算当前 token 与前面所有 token 的注意力。如果每次都重新计算前面 token 的 K、V，计算量会随序列长度线性增长，导致 $O(n^2)$ 的复杂度。

KV Cache 的思想是：在生成过程中，将已经计算过的 K、V 缓存起来，后续生成时只需计算新 token 的 Q、K、V，并利用缓存的 K、V 进行注意力计算，从而避免重复计算。这样，每步生成的计算量只与当前 token 有关（加上注意力分数计算），大幅提升推理速度。

1.3 缓存粒度的扩展

KV Cache 是针对单个推理过程内部的优化。如果多个请求具有相同的前缀（例如相同的系统提示词），那么这些请求在前缀部分的 K、V 计算结果是完全相同的。如果能够在请求之间共享这些 K、V，就能进一步减少计算量。

2. 会话缓存的需求与挑战

在对话式 AI 中，每次用户发送新消息时，应用需要将整个对话历史（通常包括系统提示词、之前的用户和助手消息）发送给模型。这些历史内容在不同轮次之间大部分是重复的：

- 系统提示词：每轮都完全相同。
- 早期对话：后续轮次中，之前的消息不会改变。

如果模型服务能够识别并复用这些重复部分的计算，就能：

- 降低延迟：省去重复的前向计算。
- 减少成本：很多云服务按处理的 token 数计费，缓存可以降低计费 token 量（例如 OpenAI 的 prompt caching 提供折扣）。
- 提升吞吐量：释放 GPU 资源处理其他请求。

挑战：

- 如何高效地判断两个请求的相同前缀？
- 如何存储和管理大量的 KV Cache（内存开销大）？
- 如何确保缓存的一致性（当前缀内容变化时缓存失效）？

3. 常见的会话缓存策略

3.1 前缀缓存（Prefix Caching / KV Cache 复用）

原理：如果两个请求的输入 token 序列共享一个共同的前缀，那么模型在处理第二个请求时，可以直接复用第一个请求已经计算好的该前缀的 KV Cache，只需要计算后缀部分。

实现：

- 缓存系统以 token 序列的前缀哈希 为键，存储对应的 KV Cache 张量。
- 当新请求到来时，计算其前缀的哈希，查找缓存中是否存在匹配的 KV。
- 如果存在，则从缓存中加载前缀的 KV，然后只对剩余 token 进行前向计算。
- 如果没有完全匹配，可以匹配最长公共前缀（部分复用）。

典型例子：

- 多个用户共享同一个系统提示词（如“你是一个编码助手”），该系统提示词的 KV 只需计算一次。
- 同一会话中，第 N 轮请求的前缀包含前 N-1 轮的所有内容，可以复用之前轮次计算好的 KV。

3.2 提示词缓存（Prompt Caching）

原理：由模型服务提供商（如 OpenAI、Anthropic、DeepSeek）在服务端实现的前缀缓存，通常自动生效，对用户透明。用户无需手动管理，但可以通过设计提示词结构来提高缓存命中率。

特点：

- 缓存粒度通常是 token 级别的连续前缀。
- 服务商会根据缓存命中情况调整计费：例如 OpenAI 对缓存命中的输入 token 收费降低（约降低 50%）。
- 缓存的生命周期有限（通常几分钟到几小时），过期后自动清除。

设计建议：

- 将稳定不变的内容放在提示词开头（如系统指令、工具定义）。
- 将会话中变化的部分放在后面。
- 对于多轮对话，保持消息顺序稳定，避免在历史中间插入新消息（会导致前缀变化，降低缓存命中）。

3.3 会话级 KV Cache 持久化

原理：在本地推理场景（如使用 vLLM、TensorRT-LLM 部署开源模型）中，可以将某个会话的完整 KV Cache 保存到磁盘或内存，下次继续同一会话时直接加载，避免重新计算整个历史。

优点：对于长会话，可以极大减少首次 token 的延迟（TTFT）。

缺点：KV Cache 占用大量内存（每个 token 每层都有 K、V 向量），持久化成本高，通常只适用于单个会话或多个活跃会话。

3.4 语义缓存（Semantic Caching）

原理：不缓存中间计算，而是缓存最终的回复结果。当新用户输入与之前的某个输入语义相似时，直接返回缓存的回复，而不调用模型。

实现：

- 将用户输入向量化（使用嵌入模型）。
- 在向量数据库中查找相似向量。
- 如果相似度超过阈值，返回对应的缓存回复。
- 否则调用模型并存储结果。

适用场景：频繁出现的相似问题（如文档问答中的常见问题）。在编码助手中，可能用于缓存常见的代码解释或错误信息回复，但不适合需要动态上下文的场景。

4. 缓存实现的技术细节

4.1 缓存键的设计

- 前缀哈希：对输入 token ID 序列计算哈希（如 SHA-256），作为缓存键。由于 token 序列可能很长，哈希可以快速匹配。
- 结构化键：对于结构化提示（如消息列表），可以构建规范化的字符串，再哈希。需要确保不同角色、工具调用等顺序稳定。

4.2 缓存存储

- 内存缓存：使用 LRU（最近最少使用）策略管理有限的内存，存储热点前缀的 KV Cache。
- 分布式缓存：在多 GPU、多节点部署中，使用 Redis 或自定义缓存服务存储 KV Cache，支持跨请求共享。
- 持久化存储：将 KV Cache 序列化到磁盘或对象存储，用于长期会话恢复（少见，因为内存开销巨大）。

4.3 缓存失效与更新

- 当前缀内容发生变化（例如用户编辑了之前的消息），缓存对应的 KV 需要失效。
- 实现中通常采用最近最少使用（LRU）或生存时间（TTL）策略自动淘汰旧缓存。
- 在会话中，如果历史消息被修改（如压缩、删除），则缓存需要重建。

4.4 与推理引擎的集成

现代推理引擎（如 vLLM、Hugging Face TGI）提供了自动前缀缓存功能：

- vLLM 的 Automatic Prefix Caching 可以自动识别相同前缀并复用 KV，无需用户干预。
- 这些引擎在调度时会将具有相同前缀的请求分组，提高缓存利用率。

5. 在 AI Harness 中的应用

在开发 Coder 类 AI Harness 时，多模型适配模块应当有意识地利用会话缓存机制：

5.1 适配器层面的优化

- 保持消息顺序稳定：在组装上下文时，确保系统提示词和工具定义始终位于最前，且内容不变，以最大化前缀缓存命中。
- 避免在历史中间插入内容：如果需要在上下文中加入检索到的代码片段，尽量放在用户消息之后或使用特殊标记，而不要改变早期历史。

- 利用提供商缓存：调用 API 时，OpenAI、Anthropic 等会自动进行前缀缓存，适配器无需额外操作，但应了解其计费规则，在成本估算中考虑缓存折扣。

5.2 与上下文压缩的协同

- 上下文压缩可能会改变历史消息（例如用摘要替换早期详细对话），这会破坏前缀缓存。因此，压缩策略需要权衡：频繁压缩会降低缓存命中率，但可以控制 token 成本。
- 一个折衷方案是：将压缩后的摘要作为一条新的消息追加，而不是直接修改原始历史，这样可以保留原始前缀的缓存，同时避免上下文过长。

5.3 本地模型部署的缓存利用

- 如果 Harness 支持本地模型（通过 Ollama、vLLM 等），可以在推理服务端启用前缀缓存。
- 适配器可以通过保持请求格式稳定来帮助服务端识别相同前缀。

5.4 用户感知

- 缓存可以显著降低响应延迟和成本，但用户通常无感知。在日志和可观测性模块中，可以记录缓存命中率，便于优化提示词结构。

6. 限制

- 内存开销：KV Cache 占用大量 GPU 显存（或 CPU 内存），缓存太多前缀可能导致 OOM。因此缓存系统通常设置容量上限和淘汰策略。
- 一致性：如果使用语义缓存，返回的回复可能不完全匹配当前上下文，需要谨慎设置相似度阈值。
- 缓存命中率：高度依赖提示词结构和请求模式。设计不良的提示词（如频繁变化的前缀）会降低命中率。
- 安全与隐私：缓存可能存储敏感信息（如用户对话），需要确保缓存存储安全，并设置合理的 TTL。